

In simple words, software engineering is a structured process for developing software that meets user requirements while ensuring quality, reliability, and cost-effectiveness.

What is Software?

Software is a collection of:

- Programs
- Procedures
- Documentation
- Data

These components work together to perform specific tasks on a computer.

Examples of Software

- Operating Systems (Windows, Linux, Android)
- MS Office
- Google Chrome
- WhatsApp
- Banking Software
- Hospital Management Systems

Need for Software Engineering

Software engineering is needed because:

- To develop high-quality software.
- To reduce development cost and time.
- To improve software reliability and security.
- To manage large and complex software projects.
- To make software easier to maintain and update.
- To satisfy customer requirements.

Objectives of Software Engineering

- Develop reliable and efficient software.
- Deliver software within budget and schedule.
- Improve software quality.
- Ensure software is easy to maintain.
- Increase customer satisfaction.
- Reduce software errors and failures.

Characteristics of Good Software

A good software system should have the following qualities:

1. Correctness

The software should perform exactly the functions required by the user.

2. Reliability

It should operate without failures under specified conditions.

3. Efficiency

The software should use CPU, memory, storage, and network resources efficiently.

4. Maintainability

It should be easy to modify, correct, and enhance.

5. Usability

The software should be easy to learn and use.

6. Security

It should protect data and prevent unauthorized access.

7. Portability

The software should work on different operating systems and hardware platforms.

Software Engineering Process

Requirement Analysis



System Design



Coding (Implementation)



Testing



Deployment



Maintenance

Requirement Analysis

The requirements of the customer are collected and analyzed.

Planning

The project schedule, budget, resources, and risks are planned.

System Design

The architecture and detailed design of the software are prepared.

Coding

Programmers write the source code using programming languages such as Java, Python, or C++.

Testing

The software is tested to identify and remove errors (bugs).

Deployment

The completed software is installed and made available to users.

Maintenance

The software is updated, improved, and corrected after deployment.

Advantages of Software Engineering

- Produces reliable and high-quality software.
- Reduces development cost.
- Improves project management.
- Helps complete projects on time.
- Makes maintenance easier.
- Improves teamwork and documentation.
- Enhances customer satisfaction.
- Reduces software failures.

Applications of Software Engineering

Software engineering is used in many fields, including:

- Banking and Financial Systems
- Hospital Management Systems
- Educational Software
- E-commerce Websites
- Mobile Applications
- Web Applications
- Railway and Airline Reservation Systems
- Artificial Intelligence

Software Requirement Analysis and Requirements

What is a Software Requirement?

A **software requirement** is a statement that describes what a software system should do or the constraints under which it must operate. Requirements define the functions, features, and performance expected from the software and serve as the foundation for software development.

What is Requirement Analysis?

Requirement Analysis is the process of gathering, studying, analyzing, documenting, and validating the needs and expectations of users and stakeholders before software development begins.

It is one of the most important phases of the **Software Development Life Cycle (SDLC)** because it ensures that the software is built according to customer needs.

Definition

Requirement Analysis is the process of identifying the customer's requirements, understanding them clearly, resolving conflicts, and documenting them in a structured format.

Objectives of Requirement Analysis

- Understand customer needs.
- Define the scope of the project.
- Identify functional and non-functional requirements.
- Reduce misunderstandings between customers and developers.
- Minimize development cost and time.
- Improve software quality.
- Create a clear Software Requirement Specification (SRS).

Steps in Requirement Analysis

Requirement Gathering



Requirement Analysis



Requirement Specification (SRS)



Requirement Validation



Requirement Management

1. Requirement Gathering

Information is collected from customers, users, and stakeholders using techniques such as:

- Interviews
- Questionnaires
- Observation
- Meetings
- Document analysis
- Prototyping

2. Requirement Analysis

The collected requirements are:

- Studied carefully.

- Prioritized.
- Checked for feasibility.
- Conflicts are identified and resolved.

3. Requirement Specification (SRS)

The approved requirements are documented in the **Software Requirement Specification (SRS)** document.

4. Requirement Validation

The requirements are reviewed to ensure they are:

- Correct
- Complete
- Consistent
- Feasible
- Testable

5. Requirement Management

Requirements are monitored and updated throughout the software development process when changes occur.

Types of Software Requirements

1. Functional Requirements

Functional requirements describe **what the system should do**. They define the features and services provided by the software.

Examples

- User registration and login.
- Search for products.
- Generate invoices.
- Process online payments.
- Print reports.

2. Non-Functional Requirements

Non-functional requirements describe **how the system should perform**. They define quality attributes and constraints.

Examples

- The system should respond within **2 seconds**.
 - The software should be available **99.9%** of the time.
 - Data must be encrypted for security.
 - The application should support **1,000 concurrent users**.
 - The interface should be easy to use.
-

Characteristics of a Good Requirement

A good requirement should be:

- **Clear:** Easy to understand.
- **Correct:** Accurately reflects user needs.
- **Complete:** Contains all necessary information.
- **Consistent:** Does not conflict with other requirements.
- **Feasible:** Can be implemented with available resources.
- **Testable:** Can be verified through testing.
- **Traceable:** Can be tracked throughout development.

Importance of Requirement Analysis

- Ensures the software meets customer expectations.
- Reduces development errors and rework.
- Improves communication between stakeholders.
- Saves development time and cost.

- Helps estimate project resources and schedule.
- Provides a strong foundation for design and implementation.

Difference Between Requirement and Requirement Analysis

Software Requirement	Requirement Analysis
Describes what the software should do.	Process of collecting, analyzing, and documenting requirements.
Defines system features and constraints.	Ensures requirements are clear, complete, and feasible.
Acts as input for software design.	Produces the Software Requirement Specification (SRS).

Software Requirement Specification (SRS)

An **SRS (Software Requirement Specification)** is a formal document that describes all the functional and non-functional requirements of a software system.

Contents of an SRS

- Introduction
- Project scope
- Functional requirements
- Non-functional requirements
- System constraints
- Assumptions and dependencies
- User interface requirements
- Performance requirements
- Security requirements

Conclusion

Requirement Analysis is a critical phase of software engineering that identifies, analyzes, and documents the needs of users before development begins. Clearly defined **functional** and **non-functional** requirements help developers build software that is reliable, efficient, and aligned with customer expectations. A well-prepared **Software Requirement Specification (SRS)** serves as the blueprint for successful software development.